

Arbitrary Precision Arithmetic

Ruhin Bhattasali
CS24BTech11054

April 2025

1 Approach

To implement arbitrary-precision arithmetic, I parse each numeric input string into an LSB-first list of digits—stripping out any decimal point for floating-point values—and then normalize by removing any leading zeros and fractional trailing zeros before performing operations on these digit arrays. For addition and subtraction, I first align their scales by appending zeros so both operands share the same fractional length; I then step through the digits one by one, propagating a carry for addition or a borrow for subtraction to produce the final result.

2 Design Considerations

- I use `List<Integer>` instead of `String` or `int[]` because it simplifies carry/borrow loops at only a modest memory cost.
- I keep a separate `boolean isNegative` flag rather than encoding the sign in the digit list, so the core loops never need to special-case negative values.
- In `AFloat` I track an integer `scale` and scale up the dividend appropriately so that division yields the correct fractional digits.
- The core digit-by-digit algorithms live in helper classes (`Addition`, `Subtraction`, `Multiplication`, `Division`), making `AInteger` and `AFloat` easier to follow with fewer lines of code.
- `AFloat` is implemented on top of `AInteger` by storing a pure-integer mantissa plus the `scale`, reusing all integer logic and avoiding code duplication.

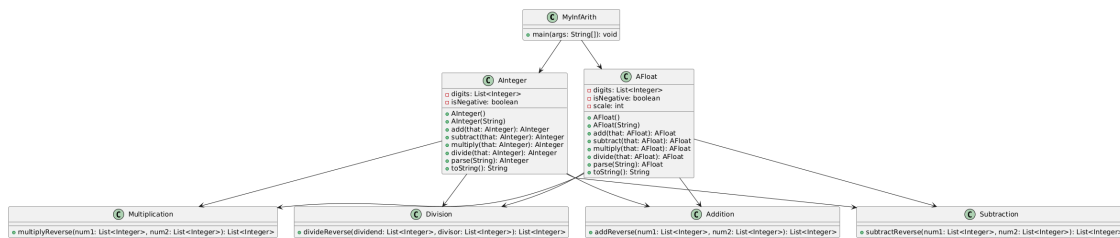


Figure 1: UML Class Diagram for Arbitrary Precision Arithmetic Project

3 Verify Approach

To verify that my code is correct and functional with accurate results I created a testcases.json file with a dictionary of the test cases. the python wrapper not only compiles, runs and tests the code but also verify

with expected results to give feedback if the answers match or not. I found that the addition, subtraction and multiplication are giving accurate results for both integer and float classes but the division function is not working correctly for float class. I made sure to handle all kinds of edge cases.

4 Limitations

- Converting the strings to digit arrays and then performing operations on them adds a extra time which can be optimised.
- Division only returns the quotient; the remainder is not exposed in `AFloat`.
- There is no support for exponent notation, scientific formatting, or advanced performance optimizations beyond the basic algorithms.

5 Key Learnings

- Writing manual digit-by-digit routines deepens understanding of how hardware bignums operate.
- Object-oriented design makes it easy to build `AFloat` on top of `AInteger` without duplicating logic.
- Careful handling of edge cases (zero, negative-zero, rounding) is as important as the core arithmetic algorithms.

6 Challenges

The main challenge was to do the division class as it was exceeding memmory limit so had to implement binary search instead of simple search algorithm. Additionally my WSL crashed while trying to configering git so had to reinstall everything in windows.